

# CPT104 Operating Systems Concepts

## Coursework 2 Task Specification

Yangbin Chen and Dongyao Jia

|                             |                                                             |
|-----------------------------|-------------------------------------------------------------|
| <b>Item</b>                 | <b>Coursework 2</b>                                         |
| <b>Weight</b>               | 30% of final mark                                           |
| <b>Work type</b>            | Individual (no team work)                                   |
| <b>Programming language</b> | C                                                           |
| <b>Platform</b>             | WebLinux                                                    |
| <b>Submission</b>           | ZIP + Report PDF + Demo video                               |
| <b>AI policy</b>            | AI is allowed as an assistant; over-reliance is not allowed |
| <b>Release date</b>         | Tuesday, May 5, 2026                                        |
| <b>Due date</b>             | Sunday, May 24, 2026, 23:59                                 |

### 0. Project Aim

In this coursework, you will build a **CPU scheduling simulator** in C. Your program will read a **workload file** describing processes and simulate multiple **CPU scheduling algorithms**. You must output a **Gantt chart** (CPU timeline), **per-process metrics**, and **overall averages**.

### 1. Expected Outcomes

- Implement and compare **FCFS**, **SJF**, **SRTF**, and **RR** scheduling.
- Correctly handle process arrivals, CPU idle periods, preemption, and completion.
- Compute and report **waiting time**, **turnaround time**, **response time**, and **CPU utilisation**.
- Produce clear **demo** evidence and pass marking **tests**.

### 2. Environment and Constraints

- Your submitted program should be able to compile and run on WebLinux.
- You may only use standard C/POSIX libraries available in WebLinux.
- Compilation must work with: `gcc -Wall -Wextra -O2` (and `-pthread` if needed).
- Your solution must be your own work; do not copy other students' code.

#### 2A. Local Development on Your Own Laptop (Allowed)

You may develop this project on your own laptop (Windows / macOS / Linux). However, the marking environment is WebLinux, so your final submission must compile and run correctly on WebLinux.

- Rule 1 (Required): Your program must compile and run on WebLinux using: `make`, `./sched --demo`, and `./sched <workload> --alg ...`
- Rule 2 (Recommended): Test on WebLinux early and often—do not wait until the last day.
- Rule 3 (Portability): Use standard C/POSIX only. Do not depend on non-standard libraries or OS-specific features.
- Rule 4 (Output format): Your final output (especially the summary block) must match the task sheet exactly.
- Windows users: use either WSL2 (Ubuntu) or compile/run directly in WebLinux. Ensure line endings are Unix-style if you edit files locally.
- macOS/Linux users: local development is usually close to WebLinux, but you must still run a final check on WebLinux.

## 2B. WebLinux Final-Check Checklist

- `make clean && make` (no compilation errors; warnings should be minimal).
- `./sched --demo` (runs successfully and prints summary blocks).
- Run at least one workload for each algorithm you implemented (FCFS, SJF, SRTF, RR).
- Confirm tie-breaking and context switch counting match your README.
- Confirm `RESULT/AVG_*` keys are printed exactly as required.

## 3. Deliverables and Submission

### 3.1 Files to submit

- Project ZIP: **CPT104\_CW2\_<StudentID>.zip**
- Report PDF (max 6 pages recommended): **CPT104\_CW2\_Report\_<StudentID>.pdf**
- Demo video (60–90 seconds): **CPT104\_CW2\_Demo\_<StudentID>.mp4**

### 3.2 ZIP folder structure (required)

CPT104\_CW2\_<StudentID>.zip

```

├── src/
│   ├── *.c
│   └── *.h
├── Makefile
├── README.md
└── (optional) tests/

```

### 3.3 Makefile requirement

Running 'make' must produce an executable named 'sched' in the project root.

`make`

`./sched --demo`

## 4. Functional Requirements (Program) (48 marks)

### 4.1 Required scheduling algorithms (minimum) (30 marks)

- FCFS (First Come First Served) — non-preemptive
- SJF (Shortest Job First) — non-preemptive
- SRTF (Shortest Remaining Time First) — preemptive
- RR (Round Robin) — preemptive, time quantum  $q$

(Bonus (up to +5%): Priority scheduling with aging OR Multilevel Feedback Queue (MLFQ).)

### 4.2 Input file format (strict) (2 marks)

Plain text, one process per line:

PID ARRIVAL BURST [PRIORITY]

Example:

```
P1 0 7
P2 2 4
P3 4 1
P4 6 4
```

- ARRIVAL is a non-negative integer (ms).
- BURST is a positive integer (ms).
- Ignore blank lines.
- Lines starting with '#' are comments (recommended).
- Invalid lines must produce a clear error message and exit non-zero.

### 4.3 Command-line interface (required) (4 marks)

Executable name: sched

```
./sched workload.txt --alg FCFS
./sched workload.txt --alg SJF
./sched workload.txt --alg SRTF
./sched workload.txt --alg RR --q 3
./sched workload.txt --alg SRTF --trace (optional)
./sched --demo
```

- --alg is required unless --demo is used.
- --q is required when --alg RR.
- --trace is optional (recommended for debugging): print step-by-step scheduling decisions.

### 4.4 Tie-breaking rules (mandatory) (4 marks)

To ensure deterministic outputs, you must apply tie-breaking in the following order:

- 1) smaller arrival time

- 2) smaller PID (lexicographic string compare)

Algorithm-specific notes:

- SJF: compare by burst time, then apply tie-break above.
- SRTF: compare by remaining time, then apply tie-break above.
- RR: ready queue is FIFO; if multiple processes arrive at the same time, enqueue them in PID order.

#### 4.5 Simulation rules (4 marks)

- You must correctly handle **CPU idle** time (print IDLE segments in the Gantt chart).
- You must handle multiple processes arriving at the same time.
- For preemptive algorithms (SRTF, RR), you must handle preemption correctly.
- You may implement time-step simulation (1ms) or event-driven simulation (recommended).

#### 4.6 Context switch counting (required) (4 marks)

A context switch occurs when the CPU changes from running one process to another. You must define whether you count IDLE→P and P→IDLE as context switches. Either choice is allowed, but you must document it in README and your implementation must match it.

### 5. Output Requirements (18 marks)

#### 5.1 Gantt chart (required) (6 marks)

Print a readable CPU timeline with time boundaries. Example:

GANTT:

0 | P1 | 7 | P2 | 11 | IDLE | 12 | P3 | 13

#### 5.2 Per-process metrics table (required) (8 marks)

For each process, print: PID, Arrival, Burst, Start, Finish, Waiting, Turnaround, Response.

Definitions (must match marking):

Turnaround = Finish - Arrival

Response = Start - Arrival

Waiting = Turnaround - Burst

#### 5.3 Machine-readable summary block (required) (4 marks)

At the end of each run, print:

RESULT: OK  
ALG=SRTF  
AVG\_WAIT=...  
AVG\_TAT=...  
AVG\_RESP=...  
CONTEXT\_SWITCHES=...  
CPU\_UTIL=...%

$\text{CPU\_UTIL} = (\text{total CPU busy time} / \text{makespan}) \times 100\%$  (makespan is end time of last segment).

## 6. Demo Requirements (6 marks)

### 6.1 --demo mode (required) (3 marks)

- Must run without any input files.
- Must contain a built-in workload (at least 4 processes).
- Must run at least two algorithms and print full outputs for each.
- Must finish within 3 seconds.

### 6.2 Demo video (required) (3 marks)

- Duration: 60–90 seconds.
- Show: `make` → `./sched --demo` → one example run with an input file (RR or SRTF recommended).
- Explain one observation (e.g., RR improves response time; SRTF reduces waiting but may increase switches).

## 7. Report Requirements (PDF) (28 marks)

Recommended length: up to 6 pages. You should write the report all by yourselves. Don't generate your report by AI. Your report must include the following sections:

### 7.1 Design and OS concepts (12 marks)

- Explain FCFS, SJF, SRTF, RR in your own words.
- Explain how you handle tie-breaking and why it matters.
- Explain your context switch counting definition.
- Describe one tricky scenario and how your program handles it (e.g., arrival during RR slice; SRTF preemption).

### 7.2 Testing strategy (12 marks)

- Describe how you tested each algorithm.

- Include edge cases: CPU idle, simultaneous arrivals, equal burst times, quantum edge cases, etc.
- Explain how you verified computed metrics.

### **7.3 AI usage declaration (required) (4 marks)**

- State what AI tools (if any) you used.
- State what AI helped with (e.g., debugging ideas, syntax).
- State how you verified correctness (tests).

## **8. Academic Integrity and AI Policy**

- This is an individual project. Sharing code is not allowed.
- AI is allowed as an assistant, but you must understand your code and pass hidden tests.
- You are allowed to use AI for:
  - Explaining concepts (FCFS/SJF/SRTF/RR, waiting/turnaround/response time)
  - Debugging help (“why is my RR queue wrong?”)
  - Syntax reminders
  - Suggesting test cases / edge cases
- You are not allowed to use AI for:
  - Submitting AI-generated code “as is” without understanding
  - Using AI to generate the full solution and only changing variable names
  - Asking AI to produce a complete project which matches the tasksheet output format
  - Sharing prompts/answers between students
  - Using AI to generate your report
- If your submission is flagged, you may be asked to explain your implementation.

## **9. Late Submission Policy**

Late submission penalties follow the module / university policy posted on the LMO.

Specifically, penalties will apply for any work submitted after the due date unless you have obtained a formal extension prior to the date for submission. The penalty applied will be 5% of the available marks for the assignment for each working day or part thereof that the assignment is late. The penalty will be capped at five working days (120 hours) from the assignment deadline. Work submitted after five working days will receive a grade of zero.